Table des matières

Cours 03 — Gestion du compte et gouvernance 1
Objectifs 1
Théorie 1
QCM — style examen (Cours 03) 9
Points clés à retenir 13

Cours 03 — Gestion du compte et gouvernance

Module 3/8 — Formation Snowflake (préparation SnowPro Core, COF-C03)

Comment lire ce module — Le domaine 2 pèse 20 %, le troisième plus lourd de l'examen. Il répond à deux questions de fond : qui a le droit de faire quoi (le contrôle d'accès, rôles et privilèges, le RBAC) et comment on protège la donnée (masquage, filtrage, chiffrement, la gouvernance). On y greffe le garde-fou des coûts : les *resource monitors*. Une idée traverse tout : chez Snowflake, la voie normale d'un droit est le rôle, pas l'utilisateur. Gravez cette phrase ; la moitié du domaine en découle. Elle souffre une exception, l'UBAC, que la section 1 présente : à l'examen comme en pratique, on raisonne en rôles, et on sait que l'autre voie existe.

Objectifs

À la fin de ce module, vous saurez :

- distinguer RBAC et DAC, et nommer les trois pièces du modèle : rôle, privilège, objet sécurisable ;
- réciter les rôles système (ORGADMIN, ACCOUNTADMIN, SECURITYADMIN, USERADMIN, SYSADMIN, PUBLIC) et leur hiérarchie d'héritage ;
- accorder l'accès proprement : `GRANT`/`REVOKE`, propriété (*ownership*), `WITH GRANT OPTION`, octrois futurs (*future grants*) ;
- situer les méthodes d'authentification : mot de passe, MFA, SSO fédéré (SAML), OAuth, paire de clés, et les politiques réseau ;
- reconnaître les outils de gouvernance : masquage dynamique (colonne), politique d'accès par ligne, tags, classification ;
- expliquer le chiffrement de bout en bout et ce qu'ajoute Tri-Secret Secure ;
- poser un resource monitor : quota, déclencheurs, actions (notifier / suspendre / suspendre aussitôt).

Théorie

© 2026 iAlexMG. Tous droits réservés.

1. Le modèle d'accès : RBAC, DAC et UBAC

Labo : `Labo_03_SNOW.md`, étape 2.

Snowflake s'appuie sur trois cadres de contrôle d'accès, complémentaires :

- RBAC (*Role-Based Access Control*, contrôle d'accès par rôle) : les privilèges sont accordés à des rôles, et les rôles sont accordés à des utilisateurs (ou à d'autres rôles). C'est la voie normale, et celle que l'examen suppose partout.
- DAC (*Discretionary Access Control*, contrôle d'accès discrétionnaire) : chaque objet a un propriétaire, et ce propriétaire décide qui peut y accéder. Créer un objet, c'est en devenir le propriétaire (via le rôle actif au moment du `CREATE`).
- UBAC (*User-Based Access Control*, contrôle d'accès par utilisateur) : un privilège peut aussi être accordé directement à un utilisateur. C'est l'exception, et elle a une condition : la documentation dit qu'un privilège accordé de cette façon n'est pris en compte que lorsque `USE SECONDARY ROLES` vaut `ALL`.

Piège — la règle d'or a une exception, mais elle reste la règle. Longtemps, « un utilisateur n'a jamais de droit en propre » était vrai sans réserve, et beaucoup de documents le disent encore. Depuis l'ajout de l'UBAC, ce n'est plus exact. En pratique et à l'examen, raisonnez en rôles : un droit qui passe par un utilisateur ne se voit pas dans la hiérarchie des rôles, ne se transmet pas, et se perd quand la personne change de poste. Sachez simplement que la voie existe, et qu'elle dépend des rôles secondaires.

Les trois pièces à nommer sans hésiter :

- Objet sécurisable — tout ce sur quoi on peut donner un droit : un entrepôt, une base, un schéma, une table, une vue, jusqu'au compte lui-même.
- Privilège — une action précise sur un objet : `USAGE`, `SELECT`, `INSERT`, `CREATE TABLE`, `OPERATE`…
- Rôle — un porte-privilèges : on lui accorde des privilèges, puis on l'accorde à des utilisateurs.

Le chaînon complet se lit privilège → rôle → utilisateur. On l'écrit avec `GRANT` (et on retire avec `REVOKE`) :

```
GRANT USAGE  ON WAREHOUSE SNOWPRO_WH TO ROLE analyste;   -- un privilège au rôle
GRANT SELECT ON TABLE ventes         TO ROLE analyste;
GRANT ROLE analyste                  TO USER bob;         -- le rôle à l'utilisateur
```

Le détour par les rôles — Il change d'échelle. Vous donnez les droits à un rôle une fois ; ensuite vous ajoutez ou retirez des gens en leur accordant (ou en leur retirant) ce rôle. Vingt analystes = un seul jeu de privilèges à maintenir, pas vingt.

Un rôle a aussi une portée, et l'examen la teste. Un rôle de compte (*account role*) reçoit des privilèges sur n'importe quel objet du compte : c'est celui qu'on manipule depuis le début, et `SYSADMIN` en est un. Un rôle de base de données (*database role*) n'en reçoit que sur une base et sur ce qu'elle contient. Il ne s'active pas tout seul, `USE ROLE` ne le prend pas : il s'accorde à un rôle de compte, qui, lui, s'active dans la session, et les droits arrivent par là.

```
CREATE DATABASE ROLE SNOWPRO_DB.lecteur;
GRANT SELECT ON ALL TABLES IN SCHEMA SNOWPRO_DB.SANDBOX TO DATABASE ROLE SNOWPRO_DB.lecteur;
GRANT DATABASE ROLE SNOWPRO_DB.lecteur TO ROLE analyste;   -- c'est ce rôle-là qui s'active
```

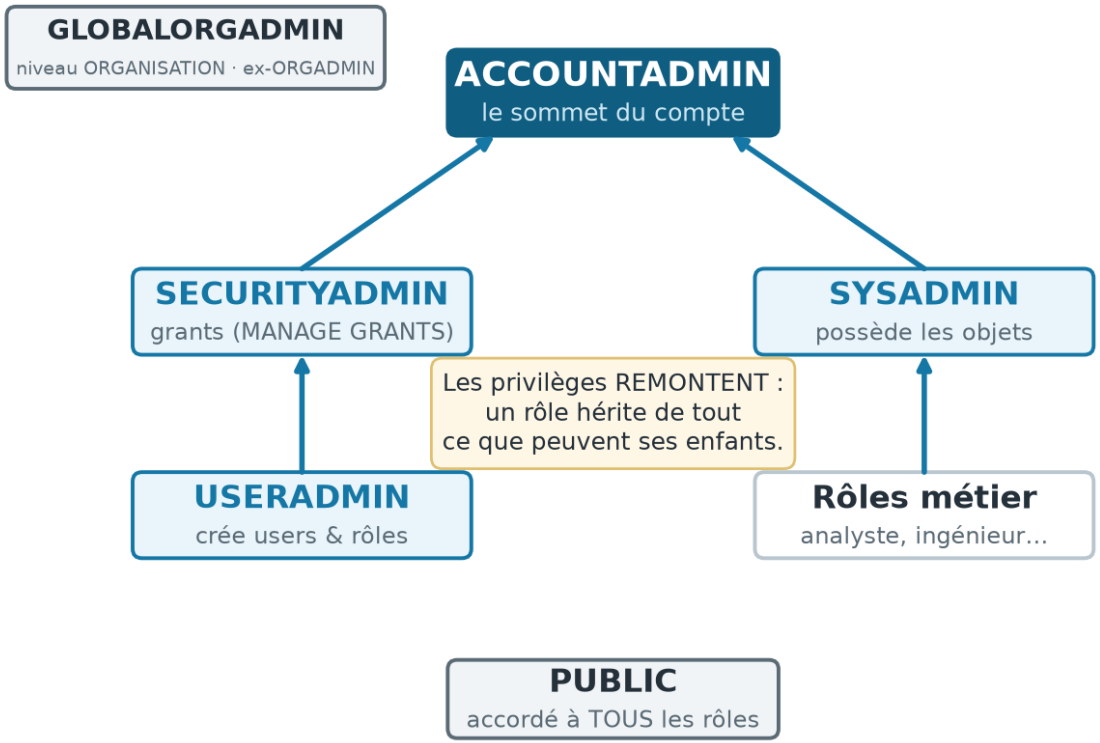

© 2026 iAlexMG. Tous droits réservés.

Pourquoi un rôle de base de données — Il voyage avec sa base. Quand on partage une base (Cours 06), les droits portés par un rôle de base de données partent avec elle, ce qu'un rôle de compte ne sait pas faire : le consommateur n'a pas vos rôles. C'est aussi la façon de déléguer l'administration d'une base sans rien donner sur le reste du compte.

Un utilisateur peut détenir plusieurs rôles, mais un seul est actif à la fois, le rôle courant, fixé par `USE ROLE` (Cours 01, section 8). Il peut aussi activer des rôles secondaires (`USE SECONDARY ROLES ALL`) pour cumuler les droits de tous ses rôles le temps d'une requête.

2. Les rôles système et leur hiérarchie

Labo : `Labo_03_SNOW.md`, étapes 1 et 2.



La hiérarchie d'héritage : ACCOUNTADMIN au sommet, qui coiffe SECURITYADMIN et SYSADMIN ; USERADMIN sous SECURITYADMIN ; les rôles métier sous SYSADMIN ; PUBLIC tout en bas, accordé à tous ; GLOBALORGADMIN à part, au niveau de l'organisation.

© 2026 iAlexMG. Tous droits réservés.

Tout compte naît avec des rôles système aux responsabilités découpées :

Rôle	Sa raison d'être
GLOBALORGADMIN	Le niveau organisation (au-dessus du compte) : créer des comptes, suivre l'usage de l'organisation. Il vit dans le compte d'organisation, à part du reste.
ORGADMIN	L'ancien rôle d'organisation. La documentation annonce son retrait dans une version future, au profit de GLOBALORGADMIN. Le connaître suffit.
ACCOUNTADMIN	Le sommet du compte, le plus puissant : facturation, tout voir, tout faire. Coiffe SECURITYADMIN et SYSADMIN. À réserver à très peu de personnes.
SECURITYADMIN	Gère la sécurité : administre les octrois partout (`MANAGE GRANTS`) et hérite de USERADMIN.
USERADMIN	Dédié aux utilisateurs et rôles : les créer et les gérer (`CREATE USER`, `CREATE ROLE`).
SYSADMIN	Crée et possède les objets : entrepôts, bases, schémas, tables… Parent recommandé de tous les rôles personnalisés.
PUBLIC	Pseudo-rôle accordé automatiquement à tous : ce qu'il possède est visible de tout le monde. Pour le vraiment partagé.

L'héritage, le point que l'examen adore. Un rôle parent hérite des privilèges des rôles qu'on lui accorde (ses enfants). D'où la chaîne : ACCOUNTADMIN hérite de SECURITYADMIN *et* SYSADMIN ; SECURITYADMIN hérite de USERADMIN. Le sommet peut donc tout ce que font ses subordonnés, jamais l'inverse.

Deux réflexes de production que l'examen teste - On ne travaille pas en ACCOUNTADMIN au quotidien : il sert à la configuration initiale et à la facturation, pas à créer des tables. Trop de pouvoir = trop de risque (et d'accidents). - On rattache ses rôles personnalisés à SYSADMIN. Ainsi SYSADMIN, qui possède les objets, garde la main sur tout ce que les rôles métier créent, et aucun objet ne devient « orphelin ».

3. Privilèges, propriété et octrois utiles

Labo : `Labo_03_SNOW.md`, étape 2.

Les conteneurs d'abord. Les objets sont emboîtés : compte → base → schéma → objet (Cours 01, section 4). Pour lire une table, il ne suffit pas d'un `SELECT` sur la table : il faut aussi le droit de traverser ses conteneurs, c'est-à-dire `USAGE` sur la base *et* sur le schéma qui la portent.

```
GRANT USAGE  ON DATABASE SNOWPRO_DB          TO ROLE analyste;
GRANT USAGE  ON SCHEMA   SNOWPRO_DB.SANDBOX  TO ROLE analyste;
GRANT SELECT ON ALL TABLES IN SCHEMA SNOWPRO_DB.SANDBOX TO ROLE analyste;
```


© 2026 iAlexMG. Tous droits réservés.

Trois notions à distinguer :

- Propriété (*ownership*) — le rôle qui crée un objet en devient propriétaire : il a tous les droits dessus et peut les distribuer. La propriété se transfère (`GRANT OWNERSHIP`).
- `WITH GRANT OPTION` — un privilège accordé ainsi permet au bénéficiaire de le ré-accorder à d'autres. Sans cette option, il peut *utiliser* le droit mais pas le *repasser*.
- Octrois futurs (*future grants*) — accorder un privilège sur les objets à venir d'un schéma (ou d'une base), pour ne pas refaire un `GRANT` à chaque nouvelle table :

```
GRANT SELECT ON FUTURE TABLES IN SCHEMA SNOWPRO_DB.SANDBOX TO ROLE analyste;
```

(Le `GRANT … ON ALL TABLES` couvre l'existant ; le `GRANT … ON FUTURE TABLES` couvre le futur. Les deux ensemble = « tout, maintenant et plus tard ».)

Déléguer la gouvernance sans donner ACCOUNTADMIN. Poser un masque, taguer une colonne ou surveiller les crédits ne demande pas le rôle le plus puissant du compte : chacun de ces gestes a son privilège, porté par le compte entier (la documentation les appelle *global privileges*), et qui s'accorde à un rôle dédié comme n'importe quel autre droit.

- `APPLY MASKING POLICY`, `APPLY ROW ACCESS POLICY` et `APPLY TAG` — poser ou retirer une politique de masquage, une politique de ligne ou un tag sur une table ou une vue, sans en être propriétaire (section 5).
- `MANAGE GRANTS` — accorder et retirer des droits sur des objets qu'on ne possède pas. C'est le privilège qui fait SECURITYADMIN, et il se borne à une base ou à un schéma quand on l'accorde sur ce conteneur-là.
- `MONITOR USAGE` — lire la consommation du compte et l'historique des entrepôts, pour qui surveille les coûts sans rien administrer d'autre.
- Sur un resource monitor (section 7), `MODIFY` change ses réglages et `MONITOR` le donne à lire. Une réserve que l'examen aime : rattacher un entrepôt à un monitor reste le privilège du seul ACCOUNTADMIN.

C'est le rôle de gouvernance dont le Labo 03 parle : sur un compte d'essai solo, il prend ACCOUNTADMIN parce que c'est le plus direct, mais en entreprise, ce sont ces privilèges-là qu'on accorde, et le sommet du compte reste rangé.

4. L'authentification : prouver qui l'on est

Le RBAC dit *ce que vous pouvez faire une fois entré* ; l'authentification vérifie *que c'est bien vous qui entrez*. Snowflake accepte plusieurs méthodes, à reconnaître :

© 2026 iAlexMG. Tous droits réservés.

Méthode	En un mot	Pour qui / quand
Mot de passe seul	l'identifiant de base, encadré par des politiques de mot de passe (longueur, complexité, expiration)	en voie de disparition : plus accepté seul pour un humain, et plus du tout pour un utilisateur non humain
MFA (*multi-facteur*)	une 2e preuve en plus du mot de passe	obligatoire pour tout humain qui s'authentifie par mot de passe
Jeton d'accès programmatique	un jeton porté par l'utilisateur, à la place du mot de passe	utilisateurs de type `SERVICE`, outils et scripts
SSO fédéré (SAML 2.0)	connexion déléguée à un fournisseur d'identité (Okta, Azure AD, ADFS…)	entreprises, authentification unique
OAuth	un jeton d'accès délégué, sans stocker de mot de passe	applications, outils de BI
Paire de clés	clés RSA : clé publique attachée à l'utilisateur, clé privée gardée côté client	automatisation (pilotes, `snow`, SnowSQL)

À retenir — le mot de passe seul est fini, et l'examen le sait. Snowflake retire l'authentification par mot de passe seul en trois phases datées, annoncées dans sa documentation. Depuis septembre 2025, tout humain qui entre dans Snowsight par mot de passe doit fournir un second facteur, sans exception. D'août à octobre 2026, la règle s'étend à tous les humains, sur toutes les surfaces. Et les utilisateurs non humains, eux, perdent le mot de passe : ils passent au type `SERVICE`, avec une paire de clés ou un jeton d'accès programmatique.

D'où deux choses à retenir. La première : le type d'un utilisateur compte. `PERSON` pour un humain, `SERVICE` pour un compte de service ; c'est ce type qui décide de ce qui lui est permis. La seconde : sur votre compte d'essai, vous vous inscrirez au second facteur dès la première connexion, et l'outil en ligne de commande ne se configurera pas par mot de passe. Le Labo 00 le fait.

Deux garde-fous d'accès qui complètent tout ça :

- Politiques réseau (*network policies*) — restreindre l'accès à des plages d'adresses IP autorisées (ou en bloquer), au niveau du compte ou d'un utilisateur.
- SCIM (*System for Cross-domain Identity Management*) — la norme qui permet à un fournisseur d'identité de provisionner automatiquement utilisateurs et rôles dans Snowflake : arrivées et départs se répercutent sans intervention manuelle.

Le piège classique — *Authentification* (prouver qui vous êtes : mot de passe, MFA, SSO…) n'est pas *autorisation* (ce que vous avez le droit de faire : le RBAC). L'examen mélange volontiers les deux : la première vous fait entrer, la seconde décide de ce que vous touchez une fois entré.

5. Gouvernance : masquer et filtrer la donnée

© 2026 iAlexMG. Tous droits réservés.

Labo : `Labo_03_SNOW.md`, étapes 3, 4 et 5.

Une fois l'accès accordé, on veut parfois montrer moins que tout selon qui regarde. Trois outils, tous à partir de l'édition Enterprise (Cours 01, section 2) : ils sont donc à votre portée si vous avez pris Enterprise à l'inscription, comme le Labo 00 le demande.

a) Masquage dynamique (*dynamic data masking*) : sécurité au niveau de la colonne. On écrit une politique de masquage : une règle qui, à la lecture et selon le rôle courant, renvoie la vraie valeur ou une version masquée. On l'attache ensuite à une colonne.

```
CREATE MASKING POLICY masque_courriel AS (val STRING) RETURNS STRING ->
  CASE WHEN CURRENT_ROLE() IN ('ACCOUNTADMIN','RH') THEN val
       ELSE '***masqué***' END;

ALTER TABLE employes MODIFY COLUMN courriel SET MASKING POLICY masque_courriel;
```

La même requête renverra le vrai courriel pour un RH, `*masqué*` pour les autres. Rien ne change dans la requête ni dans les données : le masquage s'applique à la volée.

b) Politique d'accès par ligne (*row access policy*) : cette fois on filtre les lignes, attachée à une table, elle décide, selon le rôle ou le contexte, quelles lignes chaque personne voit (par exemple, un directeur régional ne voit que sa région).

c) Tags (*object tagging*) — des paires clé–valeur collées aux objets (`sensibilite = 'PII'`, pour *Personally Identifiable Information*, les renseignements personnels qui permettent d'identifier quelqu'un : nom, courriel, numéro d'assurance sociale). Ils servent à classer, à suivre les coûts, et surtout à alimenter le masquage par tag : associer une politique de masquage à un tag, puis taguer les colonnes, et toutes les colonnes portant ce tag sont masquées automatiquement, sans les traiter une à une. La classification des données peut même proposer ces tags en repérant seule les colonnes sensibles.

Masquer une colonne vs filtrer des lignes — *Masquage* = la colonne est là, mais sa valeur est brouillée (le courriel devient `***`). *Politique par ligne* = certaines lignes disparaissent du résultat. Colonne ⇒ masque ; ligne ⇒ filtre. L'examen oppose souvent les deux.

6. Le chiffrement

Snowflake chiffre tout, automatiquement, sans réglage :

- De bout en bout : les données sont chiffrées au repos (*at rest*, stockage en AES-256) et en transit (*in transit*, TLS). À aucun moment elles ne circulent ou ne dorment en clair.
- Modèle de clés hiérarchique : une clé racine chiffre les clés de compte, qui chiffrent les clés de table, qui chiffrent les clés de fichier. Chaque étage protège le suivant (« clés qui chiffrent des clés »).
- Rotation automatique des clés, et re-chiffrement (*rekeying*) optionnel des anciennes données avec de nouvelles clés (Enterprise+). Vous n'avez rien à administrer.

© 2026 iAlexMG. Tous droits réservés.

Tri-Secret Secure ajoute un étage : une clé gérée par le client, gardée dans le KMS de son cloud (*Key Management Service*, le service de gestion des clés du fournisseur), est combinée à celle de Snowflake pour former la clé qui déverrouille les données. Le client garde ainsi un interrupteur : révoquer sa clé rend les données illisibles, même pour Snowflake. C'est du Business Critical et plus (Cours 01, section 2), donc pas sur un compte d'essai.

7. Maîtriser les coûts : les resource monitors

Labo : `Labo_03_SNOW.md`, étape 6.

L'entrepôt est le poste de coût « à l'usage » (Cours 01, section 5). Un resource monitor le surveille et agit quand la consommation dérape.

- Réglages : un quota de crédits (`CREDIT_QUOTA`) + une fréquence de remise à zéro (`DAILY`, `WEEKLY`, `MONTHLY`, `YEARLY`, `NEVER`).
- Déclencheurs (*triggers*), exprimés en % du quota, lançant une action :
  - `NOTIFY` — seulement alerter (courriel/notification) ; rien n'est coupé ;
  - `SUSPEND` — suspendre l'entrepôt en laissant finir les requêtes en cours ;
  - `SUSPEND_IMMEDIATE` — couper tout de suite, y compris les requêtes en cours.
- Portée : un monitor s'assigne au compte entier (un seul possible) ou à des entrepôts précis (un entrepôt = au plus un monitor).
- Rôle requis : ACCOUNTADMIN (le privilège est délégable).

```
CREATE RESOURCE MONITOR garde_fou WITH
  CREDIT_QUOTA = 100 FREQUENCY = MONTHLY START_TIMESTAMP = IMMEDIATELY
  TRIGGERS ON 80  PERCENT DO NOTIFY
           ON 100 PERCENT DO SUSPEND
           ON 110 PERCENT DO SUSPEND_IMMEDIATE;

ALTER WAREHOUSE SNOWPRO_WH SET RESOURCE_MONITOR = garde_fou;
```

Ce qu'un resource monitor ne fait pas — Il ne surveille que les crédits de calcul des entrepôts, pas le coût du stockage, ni certaines dépenses serverless (Snowpipe, tâches serverless…), qui relèvent d'autres garde-fous (les *budgets*). L'examen aime cette limite.

8. Les traces : l'historique d'accès et le Trust Center

Gouverner ne s'arrête pas aux règles posées : il faut savoir qui a lu quoi, et vérifier que le compte ne dérive pas. Deux outils pour ça, tous deux à reconnaître.

`SNOWFLAKE.ACCOUNT_USAGE.ACCESS_HISTORY` garde un an d'accès aux objets, tables, vues et colonnes comprises. Chaque ligne porte la requête et son auteur, puis trois listes en JSON : les objets lus directement, les objets de base derrière eux (une vue renvoie aux tables qu'elle interroge) et les objets modifiés. C'est la vue qui répond à « qui a touché à cette colonne, et quand », là où le `QUERY_HISTORY` du Cours 05 ne dit que la requête. Elle demande l'édition Enterprise ou mieux.

© 2026 iAlexMG. Tous droits réservés.

Le Trust Center, dans Snowsight, regarde le compte du point de vue du risque. Il le passe à des scanners, et ce qu'ils trouvent devient un constat (*finding*) : l'authentification à deux facteurs qui n'est pas imposée aux humains, un rôle qui en peut trop, trop de comptes d'administration, des utilisateurs inactifs. Un scanner qui ne trouve rien ne rend rien, et les constats se suivent par notification.

À retenir — Les deux ne répondent pas à la même question. `ACCESS_HISTORY` est un journal : il dit ce qui s'est passé, après coup. Le Trust Center est un examen de conformité : il dit ce qui cloche dans la configuration, maintenant.

QCM — style examen (Cours 03)

Comment répondre — Faites les dix questions avant d'ouvrir le corrigé, qui commence à la page suivante. Une question à choix multiple dit combien de réponses cocher, comme à l'examen. Comptez une minute par question.

1. Un analyste arrive dans l'équipe. Vous accordez `SELECT` sur toutes les tables du schéma au rôle qu'il détient, et il ne voit toujours rien.
    a. Il lui faut ACCOUNTADMIN : un `SELECT` seul ne suffit jamais.
    b. Il lui manque `USAGE` sur la base et sur le schéma qui portent ces tables.
    c. Le `SELECT` doit être accordé à l'utilisateur, pas à son rôle.
    d. Une politique de masquage cache les tables tant qu'on ne l'a pas levée.
2. Vous vous apprêtez à partager une base à un autre compte, et vous voulez que les droits de lecture voyagent avec elle, sans rien reconstruire chez le consommateur.
    a. Un rôle de compte, que vous accorderez au consommateur.
    b. Le rôle PUBLIC, puisqu'il est accordé à tout le monde.
    c. Un octroi futur sur le schéma, qui suivra les objets.
    d. Un rôle de base de données, qui vit dans la base et part avec elle.
3. On vous demande de retirer à une administratrice le droit de créer des utilisateurs. Elle détient SECURITYADMIN, et vous lui retirez USERADMIN.
    a. Insuffisant : SECURITYADMIN hérite de USERADMIN, elle garde donc le droit.
    b. Suffisant : USERADMIN est le seul rôle capable de créer un utilisateur.
    c. Insuffisant : seul ACCOUNTADMIN crée des utilisateurs, c'est lui qu'il faut retirer.
    d. Suffisant dès qu'elle ouvre une nouvelle session.

© 2026 iAlexMG. Tous droits réservés.

4. *(choix multiple, sélectionnez-en DEUX)* Un rôle a reçu `SELECT ON ALL TABLES IN SCHEMA`. Le lendemain, une table neuve apparaît dans ce schéma et le rôle ne la voit pas. Deux gestes règlent le cas, celui d'aujourd'hui et ceux de demain.
    a. `GRANT SELECT ON FUTURE TABLES IN SCHEMA … TO ROLE …`
    b. Rejouer `GRANT SELECT ON ALL TABLES IN SCHEMA … TO ROLE …`
    c. Réaccorder le privilège `WITH GRANT OPTION`, qui couvre les objets à venir.
    d. Transférer la propriété du schéma à ce rôle, ce qui lui donne tout ce qui s'y crée.
5. Un script de chargement doit se connecter chaque nuit, sans personne devant l'écran.
    a. Un utilisateur de type `PERSON`, avec un second facteur par message texte.
    b. Le SSO fédéré en SAML, qui délègue l'authentification au fournisseur d'identité.
    c. Un utilisateur de type `SERVICE`, avec une paire de clés RSA ou un jeton d'accès programmatique.
    d. Un mot de passe long, renouvelé tous les 90 jours par une politique de mot de passe.
6. Deux demandes arrivent ensemble : chaque directeur régional ne doit voir que les lignes de sa région, et la colonne du numéro d'assurance sociale doit s'afficher brouillée pour tous sauf les ressources humaines.
    a. Deux politiques de masquage, une par demande.
    b. Une politique d'accès par ligne pour les deux, avec une condition sur le rôle courant.
    c. Une vue par région, et une colonne calculée pour le numéro.
    d. Une politique d'accès par ligne sur la région, et une politique de masquage sur la colonne.
7. Le service de conformité veut pouvoir rendre les données illisibles, y compris pour Snowflake, en révoquant une clé qu'il détient lui-même.
    a. Le chiffrement au repos en AES-256, déjà actif sur toutes les éditions.
    b. Tri-Secret Secure, qui combine une clé du client à celle de Snowflake, en Business Critical et plus.
    c. La rotation automatique des clés, à activer à partir d'Enterprise.
    d. Une politique réseau qui n'autorise plus les adresses de Snowflake.
8. Un entrepôt de test a englouti le quota du mois pendant la nuit. La prochaine fois, vous voulez que tout s'arrête net au seuil, requêtes en cours comprises.

© 2026 iAlexMG. Tous droits réservés.

a. Un déclencheur `SUSPEND_IMMEDIATE` sur le resource monitor.

b. Un déclencheur `SUSPEND`, qui coupe net et vide la file d'attente.

c. Un déclencheur `NOTIFY` à 100 %, qui suspend après trois alertes sans réponse.

d. `AUTO_SUSPEND` réglé à zéro seconde sur l'entrepôt.

9. *(choix multiple, sélectionnez-en DEUX)* Deux de ces affirmations décrivent exactement la portée d'un resource monitor.

   a. Il surveille le stockage autant que le calcul.

   b. Il surveille les crédits de calcul consommés par les entrepôts.

   c. Un entrepôt peut porter plusieurs monitors, un par déclencheur.

   d. Les dépenses serverless, Snowpipe par exemple, lui échappent et relèvent des budgets.

10. Deux questions tombent le même matin : qui a lu la colonne des salaires le mois dernier, et pourquoi le compte n'impose toujours pas le second facteur aux humains.

    a. `QUERY_HISTORY` répond aux deux : elle porte la requête et l'état du compte.

    b. Le Trust Center répond aux deux : ses scanners gardent l'historique des accès.

    c. `ACCESS_HISTORY` pour la lecture de la colonne, le Trust Center pour le constat de configuration.

    d. `INFORMATION_SCHEMA` pour la lecture, `ACCOUNT_USAGE` pour la configuration.

© 2026 iAlexMG. Tous droits réservés.

Corrigé expliqué — Cours 03

À lire après avoir répondu. Chaque réponse nomme la section à relire : ce que vous ratez ici vous dit quoi reprendre.

1. Réponse b. Les objets sont emboîtés, et il faut le droit de traverser les conteneurs avant d'atteindre la table : `USAGE` sur la base, `USAGE` sur le schéma, puis `SELECT`. L'option a remplace un droit précis par le rôle le plus puissant du compte, ce qu'on ne fait pas. L'option c décrit la voie de l'UBAC, qui reste l'exception et ne compte que si les rôles secondaires sont à `ALL`. L'option d invente un masquage qui, de toute façon, brouille des valeurs et ne cache pas une table (sections 1 et 3).

2. Réponse d. Un rôle de base de données porte des droits sur une seule base, et il voyage avec elle quand on la partage, là où vos rôles de compte n'existent pas chez le consommateur. L'option a suppose que l'autre compte connaît vos rôles. L'option b donnerait la base à tout le monde. L'option c couvre les objets à venir, pas le passage d'un compte à l'autre (section 1).

3. Réponse a. SECURITYADMIN hérite de USERADMIN, donc il peut tout ce que fait USERADMIN, y compris créer des utilisateurs : lui retirer USERADMIN ne change rien. Les options b et c se trompent sur les rôles capables de créer un utilisateur, et l'option d prête à une session le pouvoir de contourner l'héritage (section 2).

4. Réponses a et b. Un `GRANT … ON ALL TABLES` ne couvre que ce qui existait au moment de la commande ; il faut le rejouer pour l'existant, et poser un `GRANT … ON FUTURE TABLES` pour ce qui viendra. L'option c décrit le droit de repasser un privilège à d'autres, pas de l'étendre aux objets futurs. L'option d donnerait tous les droits sur le schéma pour régler une question de lecture (section 3).

5. Réponse c. Un accès sans personne devant l'écran est un utilisateur de type `SERVICE`, et il s'authentifie par paire de clés ou par jeton d'accès programmatique : le mot de passe ne lui est plus permis. L'option a range un automate dans le type des humains et lui impose un second facteur qu'aucun script ne saura fournir. L'option b vise les connexions humaines. L'option d s'appuie sur ce qui disparaît (section 4).

6. Réponse d. Ligne et colonne sont deux outils distincts : la politique d'accès par ligne filtre les lignes visibles, la politique de masquage brouille la valeur d'une colonne. Les options a et b appliquent un seul outil aux deux besoins. L'option c se rabat sur des objets à maintenir à la main, alors que les politiques s'appliquent à la volée et suivent le rôle courant (section 5).

7. Réponse b. Tri-Secret Secure combine la clé du client, gardée dans le KMS de son cloud, à celle de Snowflake : révoquer la sienne rend les données illisibles pour tout le monde. C'est du Business Critical et plus, donc hors du compte d'essai. L'option a décrit un chiffrement réel, mais dont Snowflake détient seul les clés. L'option c change les clés sans en donner la garde. L'option d bloque un accès réseau, pas la lecture des données (section 6).

8. Réponse a. Seul `SUSPEND_IMMEDIATE` coupe les requêtes en cours ; `SUSPEND` laisse finir ce qui tourne, ce qui écarte l'option b, et `NOTIFY` se contente d'alerter, ce qui écarte l'option c. L'`AUTO_SUSPEND` de l'option d endort un entrepôt inactif : il ne surveille aucun quota (section 7).

9. Réponses b et d. Un resource monitor ne regarde que les crédits de calcul des entrepôts, et laisse de côté le stockage comme les dépenses serverless, qui relèvent des budgets. L'option a lui prête le stockage. L'option c se trompe de compte : un entrepôt porte au plus un monitor, et c'est le monitor qui porte plusieurs déclencheurs (section 7).

© 2026 iAlexMG. Tous droits réservés.

10. Réponse c. `ACCESS_HISTORY` est un journal : il garde un an d'accès aux objets, colonnes comprises, et répond à « qui a lu quoi ». Le Trust Center est un examen de configuration : ses scanners rendent des constats sur l'état du compte, par exemple un second facteur qui n'est pas imposé. `QUERY_HISTORY` ne dit que la requête, ce qui écarte l'option a, et aucun des deux outils ne fait le travail de l'autre, ce qui écarte les options b et d (section 8).

Points clés à retenir

- Règle d'or : un utilisateur n'a aucun droit en propre. RBAC = privilège → rôle → utilisateur ; DAC = chaque objet a un propriétaire qui accorde l'accès. On octroie avec `GRANT`, on retire avec `REVOKE`.
- Rôles système : ORGADMIN (organisation), ACCOUNTADMIN (sommet), SECURITYADMIN (grants, `MANAGE GRANTS`), USERADMIN (users & rôles), SYSADMIN (objets), PUBLIC (tous). Héritage : ACCOUNTADMIN ⊃ {SECURITYADMIN, SYSADMIN} ; SECURITYADMIN ⊃ USERADMIN.
- Octrois : `USAGE` sur les conteneurs (base, schéma) avant d'atteindre l'objet ; propriété transférable ; `WITH GRANT OPTION` pour re-donner ; `FUTURE` pour les objets à venir.
- Authentification (≠ autorisation) : mot de passe (+ politiques), MFA (Duo), SSO fédéré (SAML), OAuth, paire de clés (automatisation) ; politiques réseau (IP) et SCIM (provisioning).
- Gouvernance (Enterprise+) : masquage dynamique = colonne masquée selon le rôle ; politique par ligne = lignes filtrées ; tags (+ masquage par tag) et classification.
- Chiffrement : de bout en bout (au repos + en transit), clés hiérarchiques, rotation automatique ; Tri-Secret Secure (clé du client) = Business Critical+.
- Deux portées de rôle : le rôle de compte, qui reçoit tout et qui s'active ; le rôle de base de données, limité à une base, qui ne s'active pas et s'accorde à un rôle de compte. Le second voyage avec sa base quand on la partage.
- Les traces : `ACCESS_HISTORY` (un an d'accès aux objets, Enterprise+) répond après coup à « qui a lu quoi » ; le Trust Center passe le compte à des scanners et rend des constats de configuration.
- Resource monitors : quota + fréquence + déclencheurs % → `NOTIFY` / `SUSPEND` (laisse finir) / `SUSPEND_IMMEDIATE` (coupe tout). Compte ou entrepôts, réservé à ACCOUNTADMIN ; ne couvre que les crédits d'entrepôt.

© 2026 iAlexMG. Tous droits réservés.